Matrix add utilization:

Synthesis(solution1)(matrix_add_csynth.rpt)

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	113	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	45	-
Register	-	-	35	-	-
Total	0	0	35	158	0
Available	2060	2800	607200	303600	0
Utilization (%)	0	0	~0	~0	0

Detail

Instance

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	25.00 ns	6.959 ns	3.12 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
221	221	5.525 us	5.525 us	221	221	none

Detail

Vector add utilization:

Synthesis(solution1)(vector_add_csynth.rpt)

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	69	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	45	-
Register	-	-	20	-	-
Total	0	0	20	114	0
Available	2060	2800	607200	303600	0
Utilization (%)	0	0	~0	~0	0

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	25.00 ns	6.959 ns	3.12 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
102	102	2.550 us	2.550 us	102	102	none

Matrix multiplication utilization:

matrix_mult_csim.log | Synthesis(solution1)(matrix_mult_csy

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	3	0	186	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	60	-
Register	-	-	47	-	-
Total	0	3	47	246	0
Available	2060	2800	607200	303600	0
Utilization (%)	0	~0	~0	~0	0

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	25.00 ns	13.569 ns	3.12 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
2221	2221	55.525 us	55.525 us	2221	2221	none

Detail

Comments:
As expected, the matrix multiplication had the largest utilization of these three functions. This is because of the algorithm requires a lot of loading into memory from different parts of the matrix, rather than dealing with the same elements as adding does. There's probably a lot of time spent finding the elements to multiply and add. It's not surprising that matrix addition has more utilization than vector addition because of the extra dimension.

Summary:
Mat add 35 FF 158 LUT
Vec add 20 FF 114 LUT
Mat mul 46 FF 246 LUT

Mat add 6.959 ns, 221 lat and int cyc, 5.525 us abs lat
Vec add 6.959 ns, 102 lat and int cyc, 2.550 us abs lat
Mat mul 13.569 ns, 2221 lat and int cyc, 55.525 us abs lat